

상자(AFChestActor) 오브젝트 풀링 타당성 및 GAS · 네트워크 복제 영향 심층 분석 보고서

언리얼 엔진 5.8 기반 FC 프로젝트 아키텍처 원칙에 입각한 액터 수명 주기, 게임플레이 어빌리티 시스템 (GAS) 상태 무결성, 그리고 네트워크 레플리케이션 채널의 엔지니어링 분석

프로젝트	분석 대상	작성 일자	문서 버전
FC (UE 5.8)	AFChestActor	2026-09-08	v1.0 (Lead Architect)

[핵심 요약 - Executive Summary]

- 결론:** 상자(`AFChestActor`)의 오브젝트 풀링 도입은 **명백한 안티패턴(Anti-Pattern)**이자 **오버엔지니어링**입니다.
- 핵심 원인 1 (낮은 빈도):** 초당 수십 개가 생성/소멸하는 투사체와 달리, 상자는 1~3분에 1~2회 스폰되는 정적 액터이므로 풀링으로 얻을 수 있는 CPU/GC 성능 이득이 0에 수렴합니다 (0.01ms 미만).
- 핵심 원인 2 (GAS 상태 오염):** 엔진의 `UAbilitySystemComponent` 는 단방향 수명 주기(BeginPlay → EndPlay) 전제로 설계되어 원자적 Reset API가 없습니다. 풀링 시 활성 GE 타이머, Modifier 체인, GameplayTag 참조 카운트가 오염됩니다.
- 핵심 원인 3 (네트워크 채널 모순):** 언리얼 NetDriver는 액터 수명과 채널 수명을 결합하고 있습니다. 풀링 시 비활성 액터 틱 오버헤드, `FRepLayout` Shadow State 비교로 인한 `OnRep_IsOpened` 이벤트 증발, 텔레포트 팝인 등의 결함이 불가피합니다.
- 권장 방향:** 현행 `SpawnActor` / `Destroy` 표준 수명 주기를 유지하고, 풀링 리소스는 투사체(`AFCProjectileBase`)에 집중하십시오.

1. 개요 및 분석 배경

FC 프로젝트는 "단일 플레이어 기반 로컬 멀티플레이어 / Listen Server 복제 아키텍처"를 근간으로 삼고 있습니다. 모든 게임플레이 상호작용은 서버 권한(Server-Authoritative) 하에 실행되며, 클라이언트는 예측(Prediction)과 복제 이벤트 수신(RepNotify)을 통해 시각적 피드백을 전달받습니다.

최근 게임플레이 최적화 검토 과정에서 "상자(AFChestActor)를 파괴(Destroy)하지 않고 오브젝트 풀에 반환 및 재사용(Object Pooling)하는 방안"이 제기되었습니다. 본 보고서는 언리얼 엔진 5.8 내부 소스코드 수준의 동작 메커니즘을 토대로, 상자 풀링이 게임플레이 어빌리티 시스템(GAS)과 네트워크 복제(Replication) 레이어에 미치는 영향을 규명합니다.

현재 AFChestActor의 구조 및 파괴 파이프라인

- 컴포넌트 구성:** `UBoxComponent` (Root), `BaseMesh`, `LidMesh`, `UAbilitySystemComponent` (ASC), `UFCAAttributeSet`
- 네트워크 상태:** `bReplicates = true`, `SetReplicateMovement(true)`, `bIsOpened` (ReplicatedUsing = `OnRep_IsOpened`)
- 파괴 시퀀스:**

```

TakeDamage (데미지 수신)
→ DestroyAndSpawnDrops (서버 권한 검증)
→ bIsOpened = true 설정 및 HideAndDisableChest (충돌 끄기, 메쉬 숨기기)
→ AFCCardPickupActor 인스턴스 스폰 (방사형 카드 드랍)
→ 사운드 & Niagara 파티클 재생
→ SetLifeSpan(DestroyDelay) 후 엔진의 Destroy() 호출 (표준 소멸)

```

2. 오브젝트 풀링의 이론적 채택 기준

오브젝트 풀링(Object Pooling)은 유용한 최적화 기법이지만, "객체의 생성 빈도"와 "객체 내부 상태의 복잡도" 사이의 엄격한 트레이드오프를 가집니다.

평가 축	풀링 고효율 대상 (예: 투사체)	풀링 저효율/위험 대상 (예: 상자)
스폰 빈도 (Frequency)	초당 10 ~ 60회 (극도로 빈번)	수 분당 1 ~ 2회 (매우 드물)
인스턴스 수명 (Lifespan)	0.2초 ~ 2초 (극단적 단기)	수 분 ~ 영구 (장기 지속)
상태 복잡도 (State Complexity)	단순 물리 벡터, 데미지 float	GAS(ASC, AttributeSet), Replicated Bool, 드랍 테이블
GC 압력 기여도	극심함 (프레임 드랍 주원인)	무시 가능 (측정 불가 수준)
상태 리셋 실패 위험	낮음 (위치와 속도만 재설정)	치명적 (체력 0 스폰, 태그 잔존, 이벤트 누락)

3. GAS (Gameplay Ability System) 관점의 영향 및 근본 원인

`AFChestActor` 는 화염구 투사체, 광역기, 마법 스킬 등의 `GameplayEffect`를 적용받을 수 있도록 `UAbilitySystemComponent` 와 `UFCAAttributeSet` 을 직접 보유합니다. 액터를 Destroy하지 않고 풀에 반환할 때 다음과 같은 치명적 상태 오염이 발생합니다.

3.1 잔류 `GameplayEffect` 및 `FTimerManager` 백그라운드 타이머 폭주

현상: 풀에 대기 중인 상자가 오프스크린에서 스스로 파괴되거나, 다음 스폰 시 이전 전투의 디버프를 안고 나타남.

- **엔진 내부 메커니즘:** 화염 스킬 피격 시 적용된 `FCGE_Ignite` 같은 지속형(Duration/Periodic) `GameplayEffect`는 `FActiveGameplayEffectsContainer` 에 등록되며, 엔진의 전역 `FTimerManager` 에 틱 타이머 핸들을 생성합니다.
- **원인:** 표준 `Destroy()` 호출 시에는 `ASC::EndPlay()` 가 호출되어 모든 활성 타이머와 이펙트를 즉각 해제합니다. 그러나 액터를 단순히 감추기만 하면 백그라운드 타이머가 계속 동작하여 보이지 않는 상자의 체력을 깎거나 콜백을 격발합니다.

3.2 `FAttributeModifier` 체인과 `CurrentValue` 계산 왜곡

현상: 상자를 새로 꺼내 체력을 100으로 리셋했으나, 스킬을 한 대만 맞아도 체력이 음수가 되어 즉사함.

- **엔진 내부 메커니즘:** GAS의 어트리뷰트는 단순 변수가 아니라 `BaseValue` 와 `FAttributeModifier` (Additive, Multiplicative, Override) 노드들이 연결된 파이프라인으로 매번 `CurrentValue` 를 계산합니다.
- **원인:** C++에서 `AttributeSet->InitHealth(100.0f)` 로 `BaseValue`만 바꾼다고 해서 기존 GE들이 결합해 둔 Modifier 체인이 자동으로 지워지지 않습니다. 결과적으로 잔존한 피해 증폭 수식 때문에 비정상적인 데미지 계산이 일어납니다.

3.3 `FGameplayTagCountContainer` 참조 카운팅 오염

현상: 풀에서 재스폰된 상자가 플레이어의 타겟팅에서 제외되거나 상호작용 불능 상태에 빠짐.

- **엔진 내부 메커니즘:** GAS의 태그 시스템은 bool 배열이 아니라 `int32` 카운터를 기반으로 하는 참조 카운팅(Reference Counting) 구조입니다. (예: `State.Dead` 태그 1개 부여 시 카운트 = 1).
- **원인:** 수동 리셋 과정에서 GE 제거와 태그 클리어의 타이밍이 1프레임이라도 어긋나면 태그 카운트가 0으로 수렴하지 않고 영구 잔류하여, 새 상자가 `State.Dead` 또는 `State.Burning` 상태로 게임에 등장합니다.

3.4 루핑 `GameplayCue`의 고스팅 (Ghosting)

- 루핑 `GameplayCue`(지속 불꽃, 충격파 오라)는 ASC의 태그 라이프사이클에 종속되어 클라이언트 뷰포트에 렌더링됩니다. 액터가 오프스크린(예: Z = -10000)으로 이동되면 클라이언트 큐 매니저가 중단 신호를 정상 수신하지 못해 맵 엉뚱한 곳에서 사운드/이펙트가 무한 재생되는 메모리 릭이 발생합니다.

[GAS의 근본 원인 - Architectural Root Cause]

언리얼 엔진의 GAS는 "액터 스폰(BeginPlay) → 전투 및 어빌리티 처리 → 액터 소멸(EndPlay)"이라는 단방향 라이프 사이클을 엄격한 전제로 설계되었습니다. `UAbilitySystemComponent` 내부에는 모든 컨테이너(ActiveGE, Attribute Modifiers, GameplayTags, AbilitySpecs, Delegate 바인딩)를 원자적(Atomic)으로 초기화하는 공식 **Reset API**가 존재하

지 않습니다. 따라서 풀링 시 개발자가 수십 가지의 내부 컨테이너를 일일이 수동으로 분해해야 하며, 이는 심각한 유지 보수 결함의 온상이 됩니다.

4. 네트워크 복제 (Networking & Replication) 관점의 영향 및 근본 원인

AFChestActor 는 bReplicates = true 이며, bIsOpened 를 통해 상자 개방 및 파괴를 전 클라이언트에 동기화하는 네트워크 액터입니다.

4.1 UActorChannel 유지 및 불필요한 서버 틱/대역폭 낭비

- 엔진 내부 메커니즘: 언리얼의 UNetDriver 는 복제 액터마다 1:1로 UActorChannel 을 개설합니다.
- 원인: 정상적인 액터 파괴 시 채널이 즉각 Close() 되지만, 풀에 액터를 방치하면 서버 NetDriver는 보이지도 않는 액터들을 매 넷 틱마다 ConsiderList 에 올리고 클라이언트와의 거리 계산 및 프로퍼티 변경 검사를 지속합니다. 즉, 성능을 아끼려던 풀링이 오히려 네트워크 코어 틱을 낭비시킵니다.

4.2 FRepLayout Shadow State Diff 메커니즘과 RepNotify 이벤트 누락

현상: 클라이언트 화면에서 상자가 파괴되었음에도 뚜껑이 닫힌 채로 서 있거나, 파괴 SFX/VFX가 전혀 재생되지 않음.

- 엔진 내부 메커니즘: 언리얼 속성 복제(FRepLayout)는 절대값을 매 프레임 브로드캐스트하는 것이 아니라, 서버의 직전 전송 값(Shadow State)과 현재 값의 차이(Diff)를 비교하여 변경이 있을 때만 패킷을 전송합니다.
- 이벤트 누락 시나리오:

```
[1] 1회차 상자 파괴: 서버 bIsOpened = true → 클라이언트 전송 → OnRep_IsOpened 정상 실행
[2] 풀 반환: 서버 bIsOpened = false → 즉시 가시성 Off 및 휴면/비활성화
[3] 클라이언트가 원거리에서 있거나 휴면 전환으로 인해 false 변경 패킷 수신 건너뛴
[4] 2회차 재스폰 및 파괴: 서버 bIsOpened = true
[5] 클라이언트 수신 판단: 클라이언트의 Shadow Buffer 기준 (true → true) 이므로
    변화가 없다고 판단하여 OnRep_IsOpened를 트리거하지 않음! → 연출 완전 누락
```

4.3 Net Dormancy와 텔레포트 팝인(Pop-in) 현상

- 비활성 풀 액터의 채널 낭비를 막으려면 SetNetDormancy(DORM_DormantAll) 를 적용해야 합니다. 그러나 재사용 시 FlushNetDormancy() 를 호출하면, 클라이언트 측에서 "좌표 이동(Teleport)"과 "가시성 켜기", "충돌 활성화" 번치(Bunch)가 서로 다른 프레임에 도착하여 이전 대기 장소에서 새 위치로 번쩍이며 날아오는 잔상 팝인이 발생합니다.

4.4 Net Relevancy (관련성 컬링)와의 자기모순

- 풀에 액터를 보관하기 위해 맵 외곽(Z = -10000)으로 치우면, NetCullDistanceSquared 에 의해 클라이언트 측 액터 채널이 강제로 닫힙니다. 이후 액터를 다시 플레이어 근처로 꺼내오면 클라이언트는 어차피 새로운 액터 채널을 열고 모든 데이터를 처음부터 다시 수신해야 합니다. 즉, "스폰 비용을 아끼기 위한 풀링"이 네트워크 레이어에서는 "새로 스폰하는 비용"과 100% 동일해지는 모순에 빠집니다.

[네트워크 복제의 근본 원인 - Architectural Root Cause]

언리얼 엔진의 네트워킹은 "액터 스폰 = 채널 개방, 액터 파괴 = 채널 폐쇄 및 상태의 완전한 종결"이라는 대원칙을 중심으로 최적화되어 있습니다. 풀링은 물리적 파괴 없이 엔티티를 은닉하므로, 엔진 표준 복제 파이프라인(채널 수명, 새 도우 버퍼 Diff, 휴면 처리, 넷 컬링)과 필연적으로 충돌합니다.

5. 종합 비교 분석: 투사체 vs 상자

동일한 풀링 기술이라도 적용 대상 액터의 성격에 따라 엔지니어링 결과는 정반대로 나타납니다.

구분	투사체 (AFCProjectileBase)	상자 (AFC Chest Actor)
스폰 빈도	초당 10 ~ 50개	1 ~ 3분에 1 ~ 2개
인스턴스 수명	0.5 ~ 2.0초	수 분 이상 또는 영구
GAS 컴포넌트	없음 (GE 클래스 포인터만 전달)	ASC 및 AttributeSet 직접 소유
네트워크 동기화	클라이언트 예측 및 로컬 렌더링	서버 권한 및 RepNotify 양방향 복제
상태 리셋 난이도	매우 낮음 (위치, 속도, 충돌체)	극도로 높음 (ASC, Modifier, Tag, RepNotify)
풀링 적용 ROI	최상 (극적인 프레임 안정화)	최악 (성능 이득 0, 버그 위험 극대화)
권장 조치	오브젝트 풀링 적극 도입	현행 SpawnActor / Destroy 유지

6. 아키텍처 대안 및 권장 설계 방안

방안 A. 일반적인 던전/로그라이크: 현행 표준 수명 주기 유지 (강력 권장 ★★★)

현재 구현된 `FC Chest Actor.cpp`의 `SpawnActor` 및 `Destroy` 구조를 그대로 유지합니다. 현대 하드웨어와 UE 5.8에서 수 분당 1~2회의 액터 생성/파괴는 단 0.05ms 미만의 단일 프레임 비용만 발생시키며, GC 프레임 드랍을 전혀 유발하지 않습니다. 엔진 표준 수명 주기를 타는 것이 가장 안정적이고 버그가 없습니다.

방안 B. 필드 파밍/리스폰 요구 시: 제자리 상태 리셋 (In-Place State Machine)

만약 상자가 필드 특정 위치에 고정되어 일정 주기마다 다시 닫히며 리스폰되는 기획이라면, "오브젝트 풀"이 아니라 액터 내부의 상태 머신(In-Place Respawn)을 적용해야 합니다:

- 액터를 풀로 보내거나 이동시키지 않고 제자리에 유지합니다.
- 파괴 연출 후 `SetActorEnableCollision(false)`, 메쉬 숨김 처리.
- 리스폰 타이머 만료 시 `bIsOpened = false` 설정, 충돌 복구, 닫힘 애니메이션 재생.
- 액터 채널이 제자리에서 유지되므로 텔레포트 팝인이나 넷 컬링 재할당 문제가 발생하지 않습니다.

방안 C. 풀링 최적화 리소스의 재배치

엔지니어링 리소스를 상자 풀링에 투입하는 대신, 실제 성능 병목이 발생하는 영역에 우선 배분하십시오:

1. 투사체 풀링: `AFCProjectileBase` (파이어볼, 화살, 마법 투사체)
2. 드랍 아이템 풀링: 상자 파괴 시 한꺼번에 쏟아져 나오는 `AFC Card Pickup Actor`
3. UI 위젯 풀링: `FC Card Reward Widget`, 데미지 플로팅 텍스트

7. 최종 권고문 (Conclusion)

상자([AFCchestActor](#))를 오브젝트 풀링하는 것은 "얻을 수 있는 성능 이익은 측정 불가능할 정도로 미미한 반면, GAS 내부 수식자/타이머 오염과 언리얼 네트워크 채널 비동기화라는 거대한 기술 부채를 떠안는 안티패턴"입니다.

따라서 본 프로젝트에서는 **상자의 오브젝트 풀링 도입을 반려하고, 엔진 표준 수명 주기를 유지할 것을 강력히 권고**합니다.